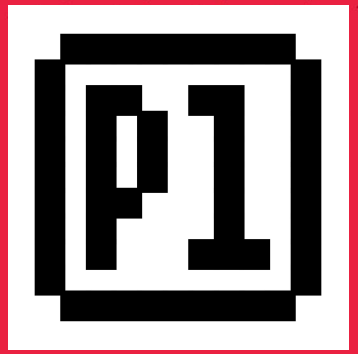
# Compilación / entorno




Mauro Fermín
mafermin@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

r33

02 / 2026

# ¿Qué herramientas vamos a usar en la materia?

Control de versiones [git]



Editor de código [vscode]



Compilador* de C [gcc]

*** Ya vamos a ver que lo que**




gitforwindows.org

cli.github.com



# GitHub

Creen una cuenta

github.com/signup

# Install

1x1



all
In
One

# Camino difícil: 1x1

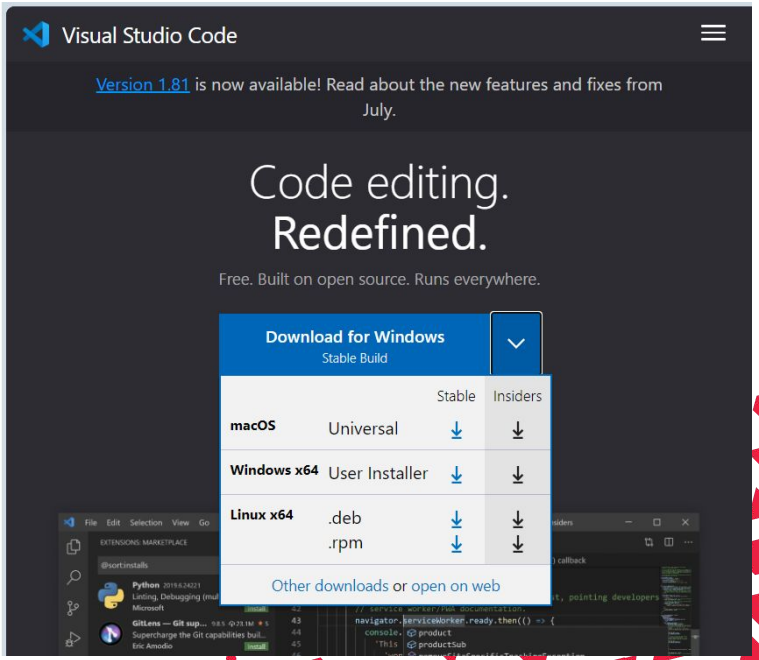


# Visual Studio Code

# Descarga de

[code.visualstudio.com](code.visualstudio.com)

(y la versión ‘stable’)



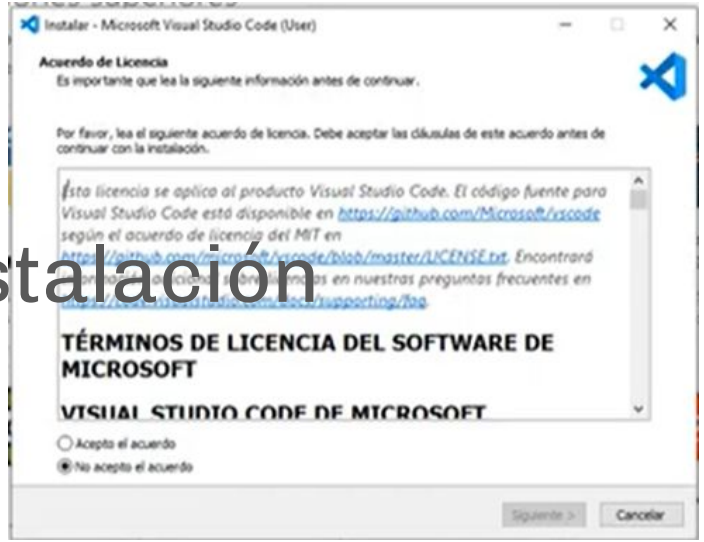
# VSCode:

2. Instalar archivo*

Aceptar términos

Seleccionar carpeta de instalación

Elegir ubicación iconos

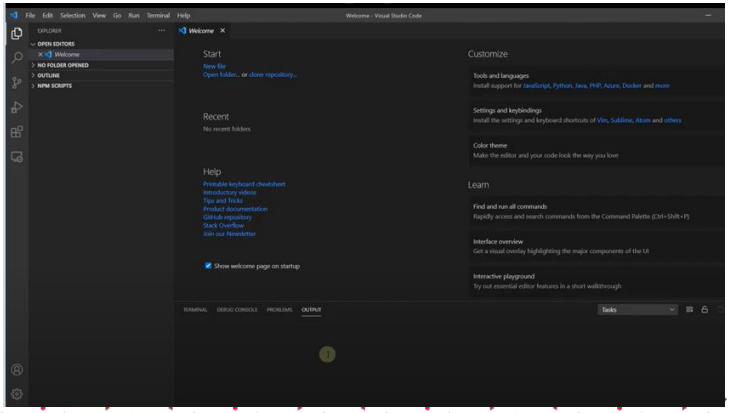



***recomendación: instalar como administrador.**

# VSCode:

## Finalizar

**Tenemos el editor funcionando pero nos falta el compilador.**




***recomendación: instalar como administrador.**

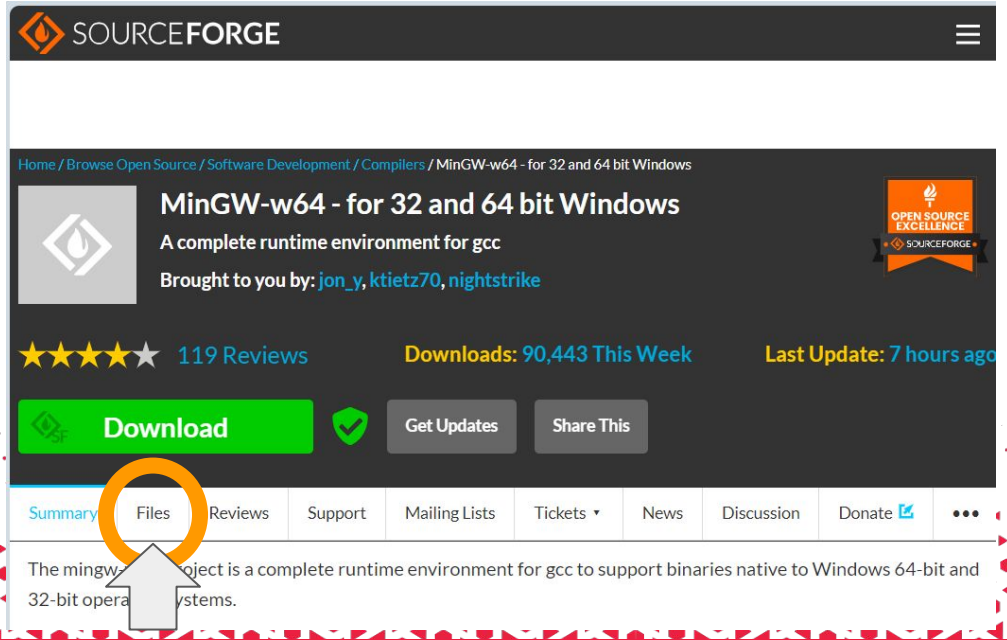
# Compilador



# GCC + Mingw:

1. Descargar: mingw-w64

2. Ir a files




https://sourceforge.net/projects/mingw-w64/

# GCC + Mingw:

## 3. Descargar archivo: x86_64-posix-sjij

**(Es posible usar el instalador, pero trae algunos problemas)**

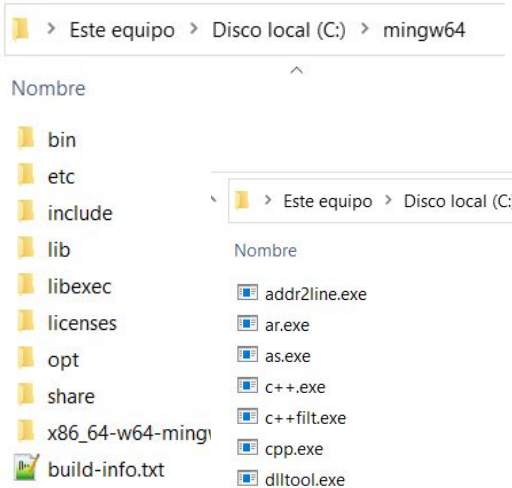


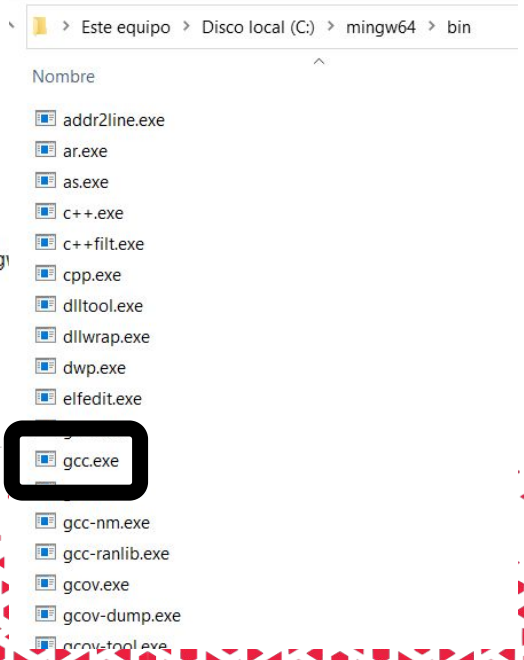

https://sourceforge.net/projects/mingw-w64/

# GCC + Mingw:

## 4. Descomprimir:

- Copiar archivo a raíz
- Descomprimir
- Verificar que se genera la carpeta: mingw64
- Y que dentro la carpeta: bin con los compiladores

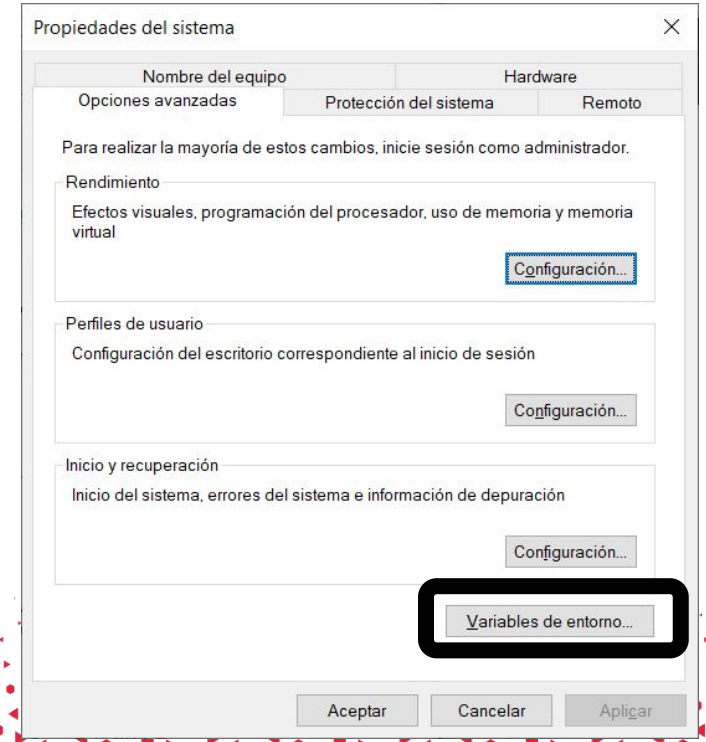


# GCC + Mingw:

## 5. Agregar ruta:

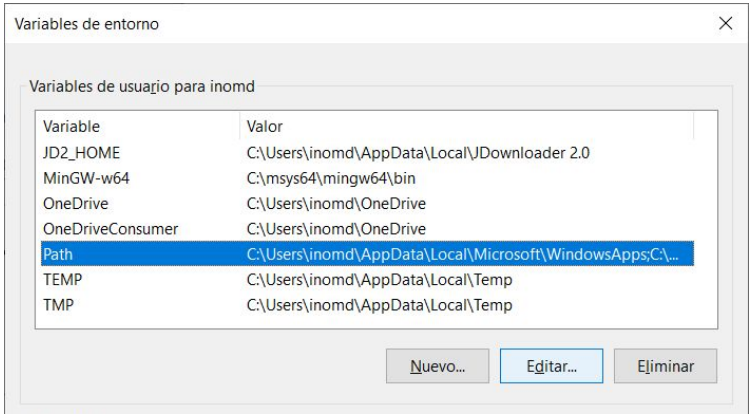
- Abrir propiedades del sistema
- Ir a variables de entorno

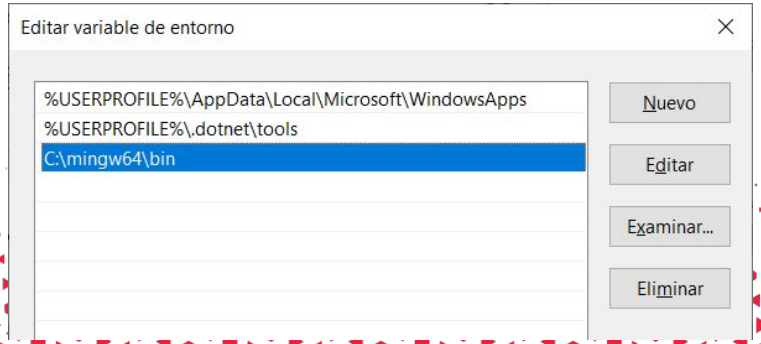


# GCC + Mingw:

## 6. Editar PATH:

- Editar variable PATH
- Agregar ruta de la carpeta bin

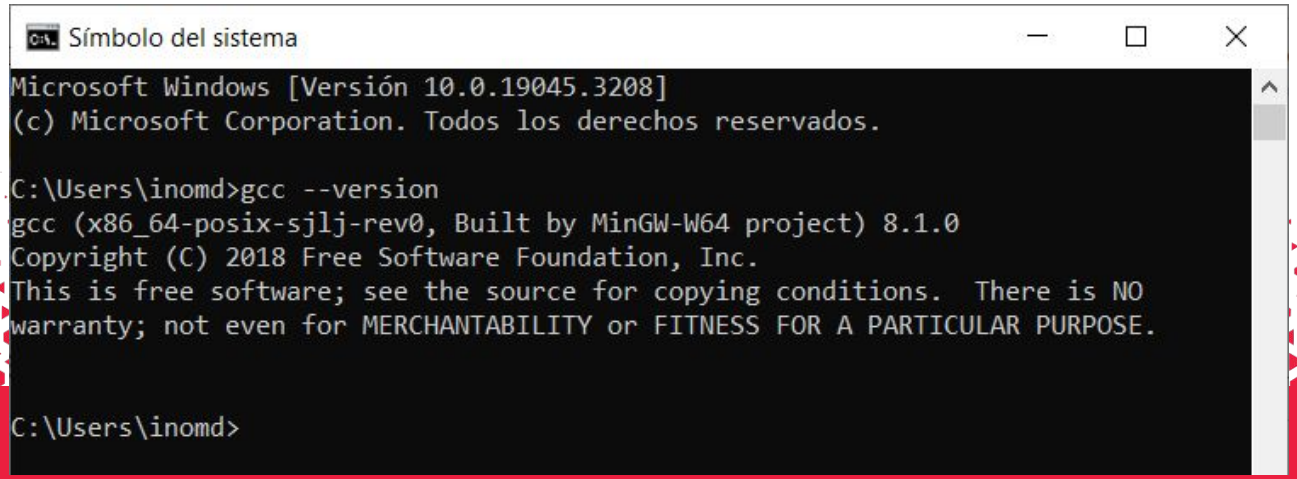


# GCC + Mingw:

## 7. Verificar:

- Abrir terminal (o símbolo del sistema CMD)
- Introducir comando: *gcc --version*

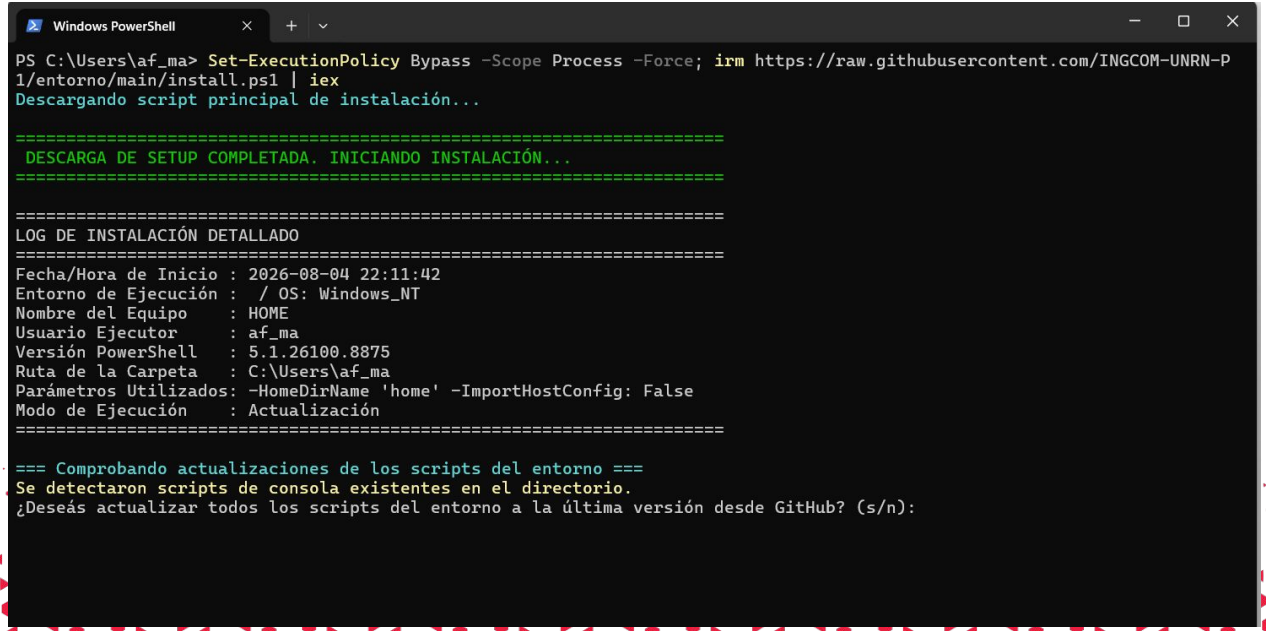


# Camino Facil:

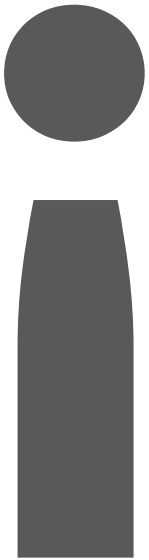
# entorno:

https://github.com/INGCOM-UNRN-P1/entorno

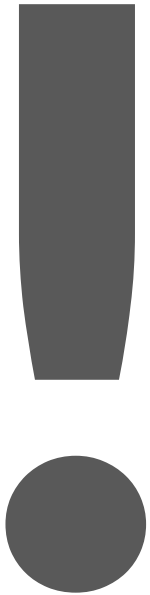
```
> Set-ExecutionPolicy Bypass -Scope Process -Force; irm
https://raw.githubusercontent.com/INGCOM-UNRN-P1/entorno/main/install.ps1 | iex
```

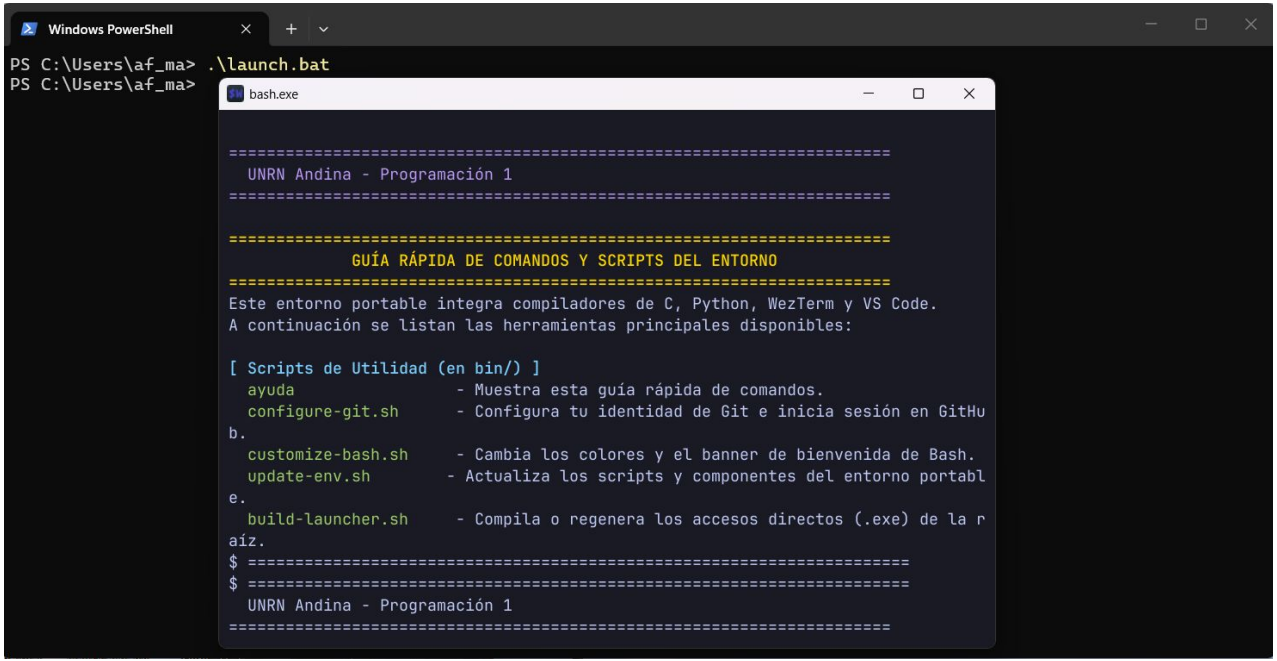




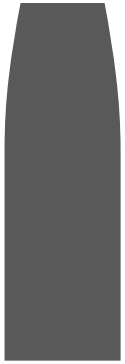
# Manos a la obra

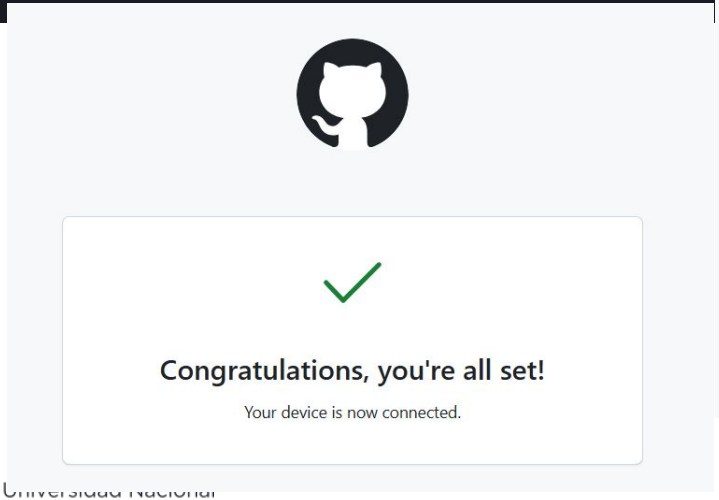


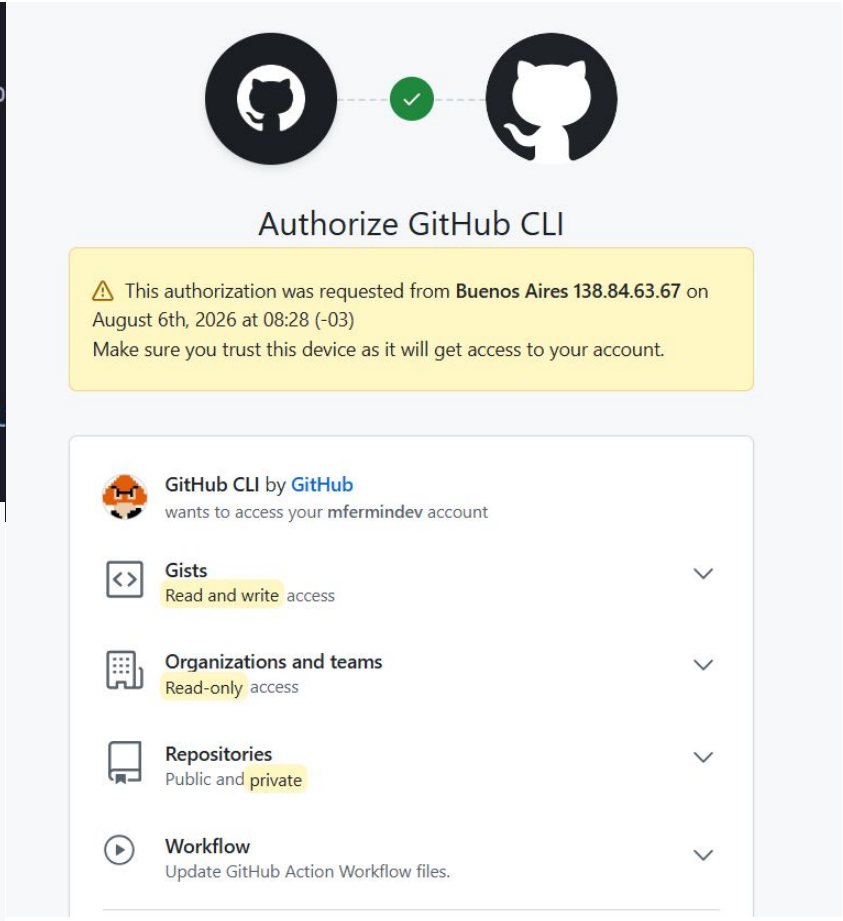

Windows PowerShell
PS C:\Users\af_ma> .\launch.bat
PS C:\Users\af_ma>
bash.exe
UNRN Andina - Programación 1
GUÍA RÁPIDA DE COMANDOS Y SCRIPTS DEL ENTORNO
Este entorno portable integra compiladores de C, Python, WezTerm y VS Code.
A continuación se listan las herramientas principales disponibles:
[ Scripts de Utilidad (en bin/) ]
ayuda - Muestra esta guía rápida de comandos.
configure-git.sh - Configura tu identidad de Git e inicia sesión en GitHub.
customize-bash.sh - Cambia los colores y el banner de bienvenida de Bash.
update-env.sh - Actualiza los scripts y componentes del entorno portable.
build-launcher.sh - Compila o regenera los accesos directos (.exe) de la raíz.
$
$
UNRN Andina - Programación 1

```
═══ Configuración de Git Portable ═══
Ingresá tu nombre para Git (Ej: Martín René): Mauro Fermin
Ingresá tu email para Git (Ej: user@gmail.com): af_mauro7@ho
Configurando almacenamiento de credenciales portable...
Git configurado con éxito.
Nombre:  Mauro Fermin
Email:   af_mauro7@hotmail.com
Destino: ~/.gitconfig (portable)

═══ Autenticación de GitHub CLI (gh) ═══
¿Querés iniciar sesión en GitHub CLI ahora? (s/n): s
? Where do you use GitHub?  [Use arrows to move, type to fil
> GitHub.com
  Other
```





# Verificando:

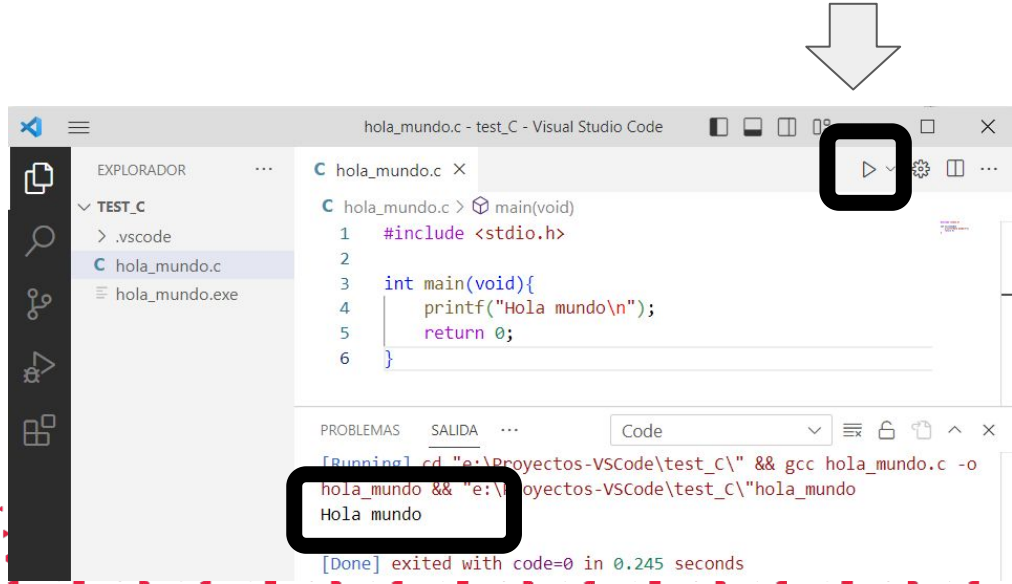
1. **Abrir VSCode**
2. **Generar archivo:** *Hola_mundo.c*
3. **Designar carpeta donde se guardan los trabajos**

# Verificando:

## 4. Copiar código

## 5. Para correr:

- Instalar extensión *code runner*
- Guardar
- Presionar botón



# Estructura de trabajo recomendada

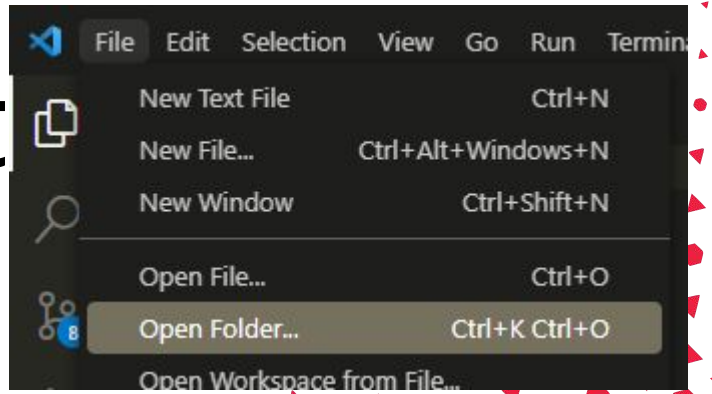
# En una ruta de directorios sin espacios, acentos y otros símbolos

# Por ejemplo

```
c:\users\usuario\p1\practicas
```

**Así dejan todas las prácticas juntas**

# Abran ese direct
# en VSCode



# EJERCICIOS

# A1

## Trabajo en equipo #1

# Resolución en Parejas (15 minutos)

La universidad requiere un pequeño módulo de validación para la inscripción a un taller avanzado. El sistema debe pedir al usuario:

1. Su **edad** (número entero).
2. Su **promedio académico** (número decimal).
3. La **cantidad de materias aprobadas** (número entero).

El programa debe calcular y mostrar por pantalla:

- Un mensaje de bienvenida indicando la edad recibida.
- Si el estudiante cumple los requisitos para inscribirse: **debe ser mayor o igual a 18 años Y tener al menos 10 materias aprobadas**.
- En caso de cumplir, mostrar: `"Inscripcion aceptada. Promedio registrado: [promedio]"`.
- En caso contrario, mostrar: `"No cumple con los requisitos mínimos."`.

# Ayudín

```c
#include <stdio.h>

int main() {
    int entero;
    float decimal;

    // Input: scanf("%d", &entero);
    // Output: printf("Texto %f", decimal);

    return 0;
}
```

# Revisión en Parejas (15 minutos)

1. **Estructura base:** ¿Incluye `#include <stdio.h>`, `int main()` y `return 0;`?
2. **Declaración de variables:** ¿Todas las variables tienen su tipo explícito (`int`, `float`) antes de usarse?
3. **Puntos y comas `;`:** ¿Casi todas las líneas de instrucción terminan en ; (excepto `#include`, `main` e `if`)?
4. **Uso de `scanf`:** ¿Utilizaron los especificadores correspondientes (`%d`, `%f`) y el símbolo `&` antes de la variable?
5. **Bloques de código:** ¿Las condiciones `if` / `else` encierran sus bloques entre llaves `{ }`?
6. **Lógica:** ¿La condición lógica (`&&`) evalúa correctamente la edad y las materias?



**Tarea del Revisor:** Si encuentran un error, **marquen la línea con un círculo** y escriban una nota al pie explicando qué fallaría al intentar compilar o ejecutar.

# Resolución en Parejas (15 minutos)

El departamento de sistemas de un club deportivo necesita una pequeña aplicación para procesar los tiempos de entrenamiento de sus atletas.

Escriban un programa en C que realice lo siguiente:

1. **Validación de entrada:** Solicite al usuario la cantidad $N$ de atletas a evaluar. El programa debe verificar que $N$ sea un número positivo ($N>0$). Si el usuario ingresa un valor menor o igual a cero, debe volver a pedirlo en bucle hasta que ingrese un valor válido (usando `while` o `do-while`).
2. **Carga de datos y acumulación:** Utilizando un lazo `for`, el programa debe pedir consecutivamente el tiempo en segundos (número flotante) de cada uno de los $N$ atletas.
3. **Cálculo de estadísticas:** Durante o al finalizar la lectura, el programa debe determinar:
    - El **tiempo promedio** del equipo.
    - La **cantidad de atletas** que superaron la marca de calificación (tiempo menor a $60.0$ segundos).
4. **Resultado:** Mostrar por pantalla el promedio general y el total de atletas calificados.

# Ayudín II

```
// Bucle while (valida mientras la condición sea verdadera)
while (condicion) { ... }

// Bucle do-while (ejecuta al menos una vez)
do {
    ...
} while (condicion);

// Bucle for (inicialización; condición; incremento)
for (int i = 0; i < limite; i++) {
    ...
}
```

# Revisión en Parejas (15 minutos)

1. **Inicialización:** ¿Los acumuladores (ej. `suma = 0.0`) y contadores (ej. `calificados = 0`) se inicializan antes de entrar a los lazos?
2. **Validación ($N>0$):** ¿El bucle `while` o `do-while` repite la solicitud correctamente cuando $N<0$?
3. **Sintaxis del `for`:** ¿El bucle `for` tiene sus tres secciones separadas por punto y coma (`;`) y la variable de control se incrementa adecuadamente?
4. **Acumulación e incremento:** ¿Se utiliza correctamente `+=` o `suma = suma + valor` y `i++` dentro de los lazos?
5. **Condición de corte:** ¿El lazo no genera un bucle infinito por olvido de actualización de variables?
6. **Promedio fuera del lazo:** ¿El cálculo del promedio se realiza **después** de completar el bucle `for` (y no en cada iteración)?

# Según la planificación

Instalación del entorno, compilación de ejemplos y uso general.
Entorno y forma de trabajo - git/github.
Bash esencial. Compilación I
Compilación de los ejercicios trabajados el lunes

# Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar